LABORATORY OBSERVATION

Course: Full Stack Web Development
Course Code: 23CM4121
Experiment No.: 4
Student Name: P.Dharani
Roll No.: 24126552261
Date: 30-08-2026

---

1. TITLE

Dynamic To-Do List Application with Interactive DOM Manipulation

2. AIM

To develop a dynamic interactive To-Do List application supporting task creation, task completion toggling, task deletion, and empty state management using JavaScript DOM manipulation.

3. OBJECTIVE

The objectives of this experiment are:

- Capture user input text and append new task items dynamically.
- Implement class list toggling (classList.toggle) to strikethrough completed tasks.
- Handle node removal (li.remove()) for deleting tasks.
- Maintain empty list message visibility dynamically.

4. THEORY

Dynamic DOM manipulation allows web applications to update content without reloading the page.

Methods like document.createElement(), element.remove(), classList.toggle(), and parent.appendChild() enable real-time UI modifications based on user interactions.

Application Flow:
Task Input -> Add Button Event -> Create List Item -> Toggle/Delete Handlers -> Dynamic List State

5. ALGORITHM / PROCEDURE

1. Build container with task input text field, Add Task button, unordered list (), and empty message paragraph.
2. Implement checkEmpty() function to display 'No tasks available' when list is empty.
3. Attach click event listener to Add Task button.
4. Read and trim input text; exit if input is empty.
5. Create item containing task title span, Complete button, and Delete button.
6. Attach click listener to Complete button to toggle 'completed' CSS class.
7. Attach click listener to Delete button to remove node and call checkEmpty().
8. Append task item to list and clear input field.

6. PROGRAM

task4.html

```
<!DOCTYPE html>
<html>
<head>
  <title>My To-Do List</title>
  <style>
    body {
      font-family: sans-serif;
      background: #f8fafc;
      padding: 30px;
      display: flex;
      justify-content: center;
    }
    .box {
      background: #ffffff;
      padding: 20px;
      border-radius: 8px;
      box-shadow: 0 2px 8px rgba(0,0,0,0.1);
      width: 400px;
    }
    h2 {
      text-align: center;
      margin-top: 0;
    }
    .row {
      display: flex;
      gap: 8px;
      margin-bottom: 15px;
    }
    input {
      flex: 1;
      padding: 8px;
    }
    button {
      padding: 8px 12px;
      cursor: pointer;
    }
    .completed {
      text-decoration: line-through;
      color: gray;
    }
    li {
      margin: 8px 0;
      display: flex;
      align-items: center;
      gap: 8px;
    }
    .task-title {
      flex: 1;
    }
    .empty-msg {
      text-align: center;
      color: gray;
    }
  </style>
</head>
<body>
  <div class="box">
    <h2>My To-Do List</h2>
    <div class="row">
```

```
      <input type="text" id="taskInput" placeholder="Enter a task...">
      <button id="addBtn">Add Task</button>
    </div>
    <ul id="taskList"></ul>
    <p id="emptyMsg" class="empty-msg">No tasks available.</p>
  </div>

  <script>
    const taskInput = document.getElementById('taskInput');
    const addBtn = document.getElementById('addBtn');
    const taskList = document.getElementById('taskList');
    const emptyMsg = document.getElementById('emptyMsg');

    function checkEmpty() {
      emptyMsg.style.display = taskList.children.length === 0 ? 'block' : 'none';
    }

    addBtn.addEventListener('click', () => {
      const text = taskInput.value.trim();
      if (!text) return;

      const li = document.createElement('li');
      const span = document.createElement('span');
      span.className = 'task-title';
      span.textContent = text;

      const compBtn = document.createElement('button');
      compBtn.textContent = 'Complete';
      compBtn.addEventListener('click', () => {
        span.classList.toggle('completed');
      });

      const delBtn = document.createElement('button');
      delBtn.textContent = 'Delete';
      delBtn.addEventListener('click', () => {
        li.remove();
        checkEmpty();
      });

      li.appendChild(span);
      li.appendChild(compBtn);
      li.appendChild(delBtn);
      taskList.appendChild(li);

      taskInput.value = '';
      checkEmpty();
    });
  </script>
</body>
</html>
```

7. CODE EXPLANATION

Code / Syntax	Explanation
document.getElementById('taskInput')	Retrieves reference to task input text box.
addBtn.addEventListener('click', ...)	Attaches click event listener for adding new tasks.

const li = document.createElement('li')	Creates new list item DOM element for task.
span.classList.toggle('completed')	Toggles strikethrough styling class for completed tasks.
li.remove()	Removes task list item node from the DOM when delete clicked.
checkEmpty()	Updates visibility of empty list notice based on task list length.

8. TEST CASES AND EXECUTION DATA

The experiment was executed with the following test cases to verify functionality:

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Initial Empty State Notice	'No tasks available' message is visible initially	'No tasks available' message is visible initially	Pass
TC-02	Task Addition	New task is added to list and empty message hides	New task is added to list and empty message hides	Pass
TC-03	Task Completion Toggle	Clicking Complete applies strikethrough style to task	Clicking Complete applies strikethrough style to task	Pass
TC-04	Task Deletion	Clicking Delete removes task item from list	Clicking Delete removes task item from list	Pass
TC-05	All Tasks Deleted Notice	Removing all tasks restores 'No tasks available' notice	Removing all tasks restores 'No tasks available' notice	Pass

9. OUTPUT

Output 1: Interactive To-Do List Application Output with Active Tasks

10. RESULT

Thus, the Dynamic To-Do List application with task creation, completion toggling, and task deletion was successfully developed and verified across all test cases.